Quiz #5 (Unit 5 - Extended): Signed & Unsigned Arithmetic, Carry & Overflow

Total Marks: 130.00

Two's Complement Numerical Bounds, Condition Flags & Machine Execution

Name: **Student ID:** **Score:** / 130.00

> **Instructions:** Answer all 10 questions. Review the technique hint provided above each problem. Adhere strictly to the designated question formats: *[Fill in the Blank]* (exact values/strings), *[Multiple Choice]* (single choice), and *[Select All That Apply]* (one or more options). Calculators are not permitted.

PART 1: 4-BIT ARITHMETIC & REGISTER RANGE EVALUATION

Question 1

Mark 10.00 out of 130.00

[FILL IN THE BLANK - ARITHMETIC VERIFICATION]

Technique / Concept Hint: 4-bit signed two's complement boundary evaluation: adding two positive numbers whose sum exceeds +7 flips the MSB to 1, producing a negative result and triggering positive overflow.

For a 4-bit signed 2's complement system, what is the result of adding 7 (`0111`) and 1 (`0001`)? Complete the explanation:

7 (0111) + 1 (0001) = 1000 (interpreted in decimal as: [ ]).

This is [ ] (an overflow / not overflow) because the valid range is [ ] to [ ] .

It is a [ ] (positive / negative / none) overflow.

Question 2

Mark 10.00 out of 130.00

[FILL IN THE BLANK - ARITHMETIC VERIFICATION]

Technique / Concept Hint: 4-bit signed negative underflow: adding two negative numbers whose sum is less than -8 wraps around to produce a positive result with MSB = 0, triggering negative overflow.

For a 4-bit signed 2's complement system, what is the result of adding –8 (`1000`) and –1 (`1111`)? Complete the explanation:

–8 (1000) + (–1) (1111) = 10111 → truncated 4-bit result: 0111 (interpreted in decimal as: [ ]).

This is [ ] (an overflow / not overflow) because the valid range is [ ] to [ ] .

It is a [ ] (positive / negative / none) overflow.

Question 3

Mark 10.00 out of 130.00

[FILL IN THE BLANK - ARITHMETIC VERIFICATION]

Technique / Concept Hint: 4-bit unsigned boundary evaluation: unsigned integers treat all 4 bits as standard positive binary powers (0 to 15); addition within this window does not trigger overflow.

For a 4-bit unsigned system, what is the result of adding 7 (`0111`) and 1 (`0001`)? Complete the explanation:

7 (0111) + 1 (0001) = 1000 (interpreted in decimal as: [ ]).

This is [ ] (an overflow / not overflow) because the valid range is [ ] to [ ] .

It is a [ ] (positive / negative / none) overflow.

Question 4

Mark 10.00 out of 130.00

[FILL IN THE BLANK - ARITHMETIC VERIFICATION]

Technique / Concept Hint: 4-bit unsigned overflow: when an unsigned addition exceeds $2^4 - 1 = 15$, a carry-out occurs and the register wraps to zero modulo 16.

For a 4-bit unsigned system, what is the result of adding 15 (`1111`) and 1 (`0001`)? Complete the explanation:

15 (1111) + 1 (0001) = 10000 → truncated 4-bit result: 0000 (interpreted in decimal as: [ ]).

The carry-out generated by the hardware is: [ ] (0 / 1).

This operation is [ ]

(an overflow / not overflow) because the sum 16 exceeds the valid range of [ ] to [ ] .

Question 5

Mark 10.00 out of 130.00

[SELECT ALL THAT APPLY]

Technique / Concept Hint: Numerical range formulas: Unsigned $[0, 2^n - 1]$ vs. Signed Two's Complement $[-2^{n-1}, 2^{n-1} - 1]$ across varying register bit widths.

Which of the following statements regarding register numerical ranges and bit interpretations are **TRUE**? (Select all that apply.)

- [ ] a. An n-bit unsigned integer register represents numbers strictly in the range 0 to $2^n - 1$.
- [ ] b. An n-bit signed two's complement register represents numbers in the range -2^{n-1} to $2^{n-1} - 1$.
- [ ] c. In an 8-bit register, the binary pattern `10000000` represents –128 in signed two's complement and 128 in unsigned binary.
- [ ] d. Signed overflow and unsigned carry are identical conditions evaluated by the exact same hardware circuit.

PART 2: 8-BIT MACHINE EXECUTION & CONDITION FLAGS

Question 6

Mark 20.00 out of 130.00

[FILL IN THE BLANK - MACHINE STATE TRACING]

Technique / Concept Hint: Dual interpretation of 127 + 1: valid unsigned representation within [0, 255] with no carry, contrasted against signed overflow exceeding +127.

In an 8-bit computer, the machine executes the addition **127 + 1**. Complete the verification:

Unsigned Check: 127 + 1 = 128, which is [ ]

(in between / not in between) 0 and 255, so it is [ ] (valid / invalid).

```
  01111111
(+) 00000001
=  [ ]
```

• Then Carry flag = [ ] (0 / 1).

Signed Check: 127 + 1 = 128, which is [ ] (greater than 127 / less than -128), so it is [ ] (valid / invalid).

• Then Overflow flag = [ ] (0 / 1).

• It is a [ ] (Positive Overflow / Negative Overflow / No Overflow).

Question 7

Mark 20.00 out of 130.00

[FILL IN THE BLANK - MACHINE STATE TRACING]

Technique / Concept Hint: Machine tracing for positive sums exceeding +127: binary addition, carry bit generation, and signed overflow detection.

In an 8-bit computer, the machine executes the addition **120 + 15**. Complete the verification:

Unsigned Check: 120 + 15 = 135, which is [ ]

(in between / not in between) 0 and 255, so it is [ ] (valid / invalid).

```
  01111000
(+) 00001111
=  [ ]
```

• Then Carry flag = [ ] (0 / 1).

Signed Check: 120 + 15 = 135, which is [ ] (greater than 127 / less than -128), so it is [ ] (valid / invalid).

• Then Overflow flag = [ ] (0 / 1).

• It is a [ ] (Positive Overflow / Negative Overflow / No Overflow).

Question 8

Mark 20.00 out of 130.00

[FILL IN THE BLANK - MACHINE STATE TRACING]

Technique / Concept Hint: Dual interpretation of 255 + 1: unsigned sum of 256 causes 8-bit truncation (Carry = 1); signed interpretation treats 0xFF as -1, giving -1 + 1 = 0 with no overflow.

In an 8-bit computer, the machine executes the addition **255 + 1**. Complete the verification:

Unsigned Check: 255 + 1 = 256, which is [ ]

(in between / not in between) 0 and 255, so it is [ ] (valid / invalid).

```
  11111111
(+) 00000001
= [ ]
```

- Then Carry flag = [ ] (0 / 1).

Signed Check: Interpret 0xFF (255) as signed decimal: [ ] .

- Signed arithmetic: (–1) + 1 = 0, which is within the range –128 to 127, so it is [ ] (valid / invalid).
- Then Overflow flag = [ ] (0 / 1).

Question 9

Mark 10.00 out of 130.00

[FILL IN THE BLANK - NEGATIVE UNDERFLOW ANALYSIS]

Technique / Concept Hint: Tracing -128 - 1 (10000000 + 11111111): generating a carry out of 1 while exceeding the minimum negative bound of -128, triggering negative overflow.

In an 8-bit computer, the machine executes **–128 – 1** computed as –128 + (–1). Complete the trace:

```
  10000000  (-128 signed)
(+) 11111111  (-1 signed)
= [ ]
```

- Then Carry flag = [ ] (0 / 1).
- Signed evaluation: –128 – 1 = –129, which is less than the minimum bound of –128.
- Then Overflow flag = [ ] (0 / 1).
- It is a [ ] (positive / negative / none) overflow.

Question 10

Mark 10.00 out of 130.00

[MULTIPLE CHOICE - SELECT ONE]

Technique / Concept Hint: Hardware overflow detection logic: comparing carry-in and carry-out of the sign bit using an XOR gate ($V = C_{in} \oplus C_{out}$).

In digital hardware ALUs, how is the **Signed Overflow flag (V)** generated during binary addition?

- a. By taking the exclusive-OR (XOR) between the carry-in to the most significant bit and the carry-out of the most significant bit ($V = C_{in} \oplus C_{out}$)
- b. By directly wiring the carry-out of the least significant bit (LSB) to the flag register
- c. By checking whether all 8 output bits are simultaneously equal to 1
- d. By checking whether the Program Counter address is aligned to a 4-byte boundary

Instructor Answer Key & Detailed Marking Scheme

Maximum Marks: 130.00

Quiz #5 (Unit 5 - Extended) - Step-by-Step Solutions & Flag Rules

---

Question 1 Solution: 4-Bit Signed Addition of 7 + 1 (10.00 Marks)

Interpreted decimal: [-8] | Status: [an overflow] | Valid range: [-8] to [7] | Overflow type: [positive]

Explanation: Adding 7 (0111_2) + 1 (0001_2) produces 1000_2. In a 4-bit signed system, 1000_2 has an MSB of 1, denoting -8. Because two positive operands yielded a negative result (and +8 exceeds +7), this is a positive signed overflow.

Question 2 Solution: 4-Bit Signed Addition of -8 + (-1) (10.00 Marks)

Interpreted decimal: [7] (or +7) | Status: [an overflow] | Valid range: [-8] to [7] | Overflow type: [negative]

Explanation: Adding -8 (1000_2) + (-1) (1111_2) = 10111_2. Discarding the 5th bit gives 0111_2 = +7. Adding two negative numbers resulted in a positive value because the sum -9 is below the lower bound of -8, representing negative signed overflow.

Question 3 Solution: 4-Bit Unsigned Addition of 7 + 1 (10.00 Marks)

Interpreted decimal: [8] | Status: [not overflow] | Valid range: [0] to [15] | Overflow type: [none]

Explanation: In an unsigned system, 1000_2 represents positive +8. Because 8 lies comfortably within the unsigned 4-bit range of 0 to 15, no overflow occurs.

Question 4 Solution: 4-Bit Unsigned Addition of 15 + 1 (10.00 Marks)

Interpreted decimal: [0] | Carry-out: [1] | Status: [an overflow] | Valid range: [0] to [15]

Explanation: Adding 15 (1111_2) + 1 (0001_2) = 10000_2. The register stores 0000_2 = 0 while the 5th bit generates a carry-out (C = 1), representing an unsigned overflow because the sum 16 exceeds 15.

Question 5 Solution: Range & Representation Concepts (10.00 Marks)

Correct Answers: a, b, c

Explanation:

- **a is TRUE:** n-bit unsigned numbers range from 0 to 2^n - 1.
- **b is TRUE:** n-bit two's complement numbers range from -2^{n-1} to 2^{n-1} - 1.
- **c is TRUE:** Pattern `10000000` represents -128 signed and +128 unsigned.
- **d is FALSE:** Carry (unsigned out-of-range) and Overflow (signed out-of-range) rely on distinct hardware detection circuits.

Question 6 Solution: 8-Bit Addition of 127 + 1 (20.00 Marks)

Unsigned: [in between] | [valid] | Binary sum: [10000000] | Carry: [0]
Signed: [greater than 127] | [invalid] | Overflow: [1] | Type: [Positive Overflow]

Explanation: 127 + 1 = 128. Unsigned: 128 ≤ 255, so no carry (C = 0). Signed: $01111111_2 + 00000001_2 = 10000000_2$ (-128), which is an invalid representation of +128, setting the overflow flag (V = 1).

Question 7 Solution: 8-Bit Addition of 120 + 15 (20.00 Marks)

Unsigned: [in between] | [valid] | Binary sum: [10000111] | Carry: [0]
Signed: [greater than 127] | [invalid] | Overflow: [1] | Type: [Positive Overflow]

Explanation: 120 + 15 = 135. In unsigned binary, 135 fits within [0, 255], yielding 10000111_2 with Carry = 0. In signed binary, two positive numbers produced an MSB of 1 (interpreted as -121), exceeding +127 and triggering Positive Overflow (V = 1).

Question 8 Solution: 8-Bit Addition of 255 + 1 (20.00 Marks)

Unsigned: [not in between] | [invalid] | Binary sum: [00000000] | Carry: [1]
Signed decimal: [-1] | Signed status: [valid] | Overflow: [0]

Explanation: $11111111_2 + 00000001_2 = 100000000_2 \rightarrow 00000000_2$ with Carry = 1. Unsigned 256 > 255 is invalid. Under signed interpretation, 11111111_2 is -1, and -1 + 1 = 0, which is valid and sets Overflow = 0.

Question 9 Solution: 8-Bit Execution of -128 - 1 (10.00 Marks)

Binary sum: [01111111] | Carry: [1] | Overflow: [1] | Type: [negative]

Explanation: Adding 10000000_2 (-128) + 11111111_2 (-1) = 101111111_2. The register stores 01111111_2 (+127) with Carry = 1. Because the true mathematical result -129 is less than -128, the signed result wrapped to positive, setting Negative Overflow (V = 1).

Question 10 Solution: Hardware Overflow Detection (10.00 Marks)

Correct Answer: a. By taking the exclusive-OR (XOR) between the carry-in to the most significant bit and the carry-out of the most significant bit (V = C_in XOR C_out)

Explanation: Signed overflow occurs if and only if the carry entering the sign bit does not equal the carry leaving the sign bit ($V = C_{in} \oplus C_{out}$).
